МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №2

з дисципліни **"НерПрофесійний Python"** на

тему

"Структури даних та функціональна декомпозиція"

Виконала:

Студентка 3 курсу

групи ФеП-32

Гуділіна Анастасія

Перевірив:

проф. Свелеба С. А.

Львів-2026

1. Мета роботи
Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю.

2. У ході роботи необхідно опанувати:
- list, tuple, set, dict;
- модуль collections (Counter, defaultdict, deque);
- list / set / dict comprehensions та генераторні вирази;
- оцінювання часової складності операцій (Big-O);
- функціональну декомпозицію;
- позиційні та іменовані аргументи, *args, **kwargs;
- lambda, closures, decorators.

3. Номер індивідуального варіанта
Варіант № 4 — Аналіз системи банківських рахунків.

---

4. Постановка задачі
Реалізувати модуль обробки структурованих даних для системи банківських рахунків.
Дані про кожен рахунок містять:
- ім'я клієнта;
- номер рахунку;
- баланс;
- тип рахунку.

Необхідно реалізувати:
1. пошук рахунку за номером;
2. групування рахунків за типом;
3. загальну суму коштів;
4. середній баланс;
5. рахунок із найбільшим балансом;
6. фільтрацію рахунків за мінімальним балансом;
7. сортування за балансом;
8. dict-index за номером рахунку.

Програма повинна містити: list, tuple, set, dict, щонайменше одну структуру з collections, list/set/dict comprehension, мінімум 6 функцій, функцію з *args, функцію з **kwargs, хоча б одну lambda, closure, decorator, а також реалізовувати пошук, фільтрацію, сортування, групування, агрегацію та оцінювання складності щонайменше трьох операцій.

---

5. Структура проєкту
Проєкт побудовано за **src-layout**:

text
```
project/
├── pyproject.toml
├── README.md
├── .gitignore
└── src/
    └── data_processor/
        ├── __init__.py
        ├── main.py
        ├── data.py
        ├── processors.py
        ├── analytics.py
        └── decorators.py
```

Елемент	Призначення
pyproject.toml	метадані пакета, конфігурація збірки
README.md	документація проєкту
.gitignore	перелік файлів, що не потрапляють у git
src/data_processor/	кореневий пакет
__init__.py	ініціалізація пакета, __all__
main.py	точка входу, демонстрація роботи
data.py	вхідні структуровані дані, генератор
processors.py	пошук, фільтрація, групування, сортування
analytics.py	агрегація, статистика, closures, benchmark
decorators.py	декоратори log_call, measure_time, repeat

---

6. Опис створених модулів

data.py

Містить:

- ACCOUNT_TYPES — tuple з допустимих типів рахунків;
- CURRENCIES — set валют;
- accounts — list[dict] — основні структуровані дані;

- generate_accounts(n) — генератор синтетичних даних для бенчмарку (використовує list comprehension).

decorators.py

- log_call(func) — логування виклику функції;
- measure_time(func) — вимірювання часу виконання;
- repeat(count) — **параметризований декоратор** (демонструє decorator + closure + *args + **kwargs).

processors.py

Функції «чистої» обробки:

- comprehensions: get_unique_types, create_account_index, create_id_index;
- фільтрація: filter_by_type, filter_by_min_balance, filter_items;
- групування: group_by_type, group_by_field;
- підрахунок: count_accounts_by_type, count_accounts_by_currency;
- пошук: search_account_linear, search_account_by_index;
- універсальне сортування: sort_items;
- pipeline: pipeline(data, *steps);
- OperationHistory — клас на базі deque.

analytics.py

- агрегація: calculate_total_balance, calculate_average_balance, balance_by_type, average_balance_by_type, balance_statistics;
- топ/антитоп: find_richest_account, find_poorest_account;
- сортування: sort_by_balance, sort_by_client;
- *args: calculate_average_values;
- **kwargs: create_record;
- closures: create_min_balance_filter, create_balance_range_filter;
- benchmark: benchmark_search.

main.py

Містить 10 демонстраційних секцій, кожна з яких ілюструє окрему вимогу завдання.

---

7. Текст основних фрагментів коду

7.1 pyproject.toml

toml

```
[project]
name = "data-processor"
version = "0.1.0"
description = "Lab 02 — banking accounts analysis (Variant 4)"
requires-python = ">=3.11"

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
```

```
[tool.setuptools.packages.find]
where = ["src"]
```

7.2 data.py — структури даних

```
python
# tuple — незмінний перелік типів рахунків
ACCOUNT_TYPES: tuple[str, ...] = (
    "current", "savings", "deposit", "credit",
)

# set — унікальні валюти
CURRENCIES: set[str] = {"UAH", "USD", "EUR"}

# list[dict] — основні дані
accounts: list[dict] = [
    {"id": 1, "client": "Anna Koval",
     "account_number": "UA0100001",
     "balance": 15230.50, "type": "current", "currency": "UAH"},
    # ...
]


def generate_accounts(n: int) -> list[dict]:
    import random
    currency_list = list(CURRENCIES)
    return [
        {
            "id": i,
            "client": f"Client_{i}",
            "account_number": f"UA{1_000_000 + i:07d}",
            "balance": round(random.uniform(100.0, 500_000.0), 2),
            "type": ACCOUNT_TYPES[i % len(ACCOUNT_TYPES)],
            "currency": currency_list[i % len(currency_list)],
        }
        for i in range(1, n + 1)
    ]
```

7.3 decorators.py

```
python
from functools import wraps
from time import perf_counter
```

```
def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[LOG ] -> {func.__name__}")
        result = func(*args, **kwargs)
        print(f"[LOG ] <- {func.__name__}")
        return result
    return wrapper


def measure_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"[TIME] {func.__name__}: {elapsed:.8f} s")
        return result
    return wrapper


def repeat(count: int):
    """Параметризований декоратор (closure + decorator)."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(count):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator
```

7.4 processors.py — фрагмент

```
python
from collections import Counter, defaultdict, deque

def get_unique_types(accounts: list[dict]) -> set[str]:
    """Set comprehension."""
    return {a["type"] for a in accounts}
```

```
def create_account_index(accounts: list[dict]) -> dict[str, dict]:
    """Dict comprehension — O(1) середній доступ."""
    return {a["account_number"]: a for a in accounts}


def filter_by_min_balance(accounts: list[dict],
                          minimum: float) -> list[dict]:
    """List comprehension."""
    return [a for a in accounts if a["balance"] >= minimum]


def group_by_type(accounts: list[dict]) -> dict[str, list[dict]]:
    """defaultdict — групування за типом."""
    grouped: defaultdict[str, list[dict]] = defaultdict(list)
    for a in accounts:
        grouped[a["type"]].append(a)
    return dict(grouped)


def count_accounts_by_type(accounts: list[dict]) -> Counter:
    return Counter(a["type"] for a in accounts)
```

7.5 analytics.py — closures та *args/kwargs**

python

```
def create_min_balance_filter(minimum: float):
    """Closure: predicate пам'ятає `minimum`."""
    def predicate(account: dict) -> bool:
        return account["balance"] >= minimum
    return predicate


def calculate_average_values(*values: float) -> float:
    if not values:
        return 0.0
    return sum(values) / len(values)


def create_record(**fields) -> dict:
    return dict(fields)
```

7.6 main.py — фрагмент pipeline

python

```
def demo_pipeline() -> None:
    result = pipeline(
        accounts,
        lambda data: filter_by_min_balance(data, 20_000.0),
        group_by_type,
    )
    print({t: len(v) for t, v in result.items()})
```

---

8. Пояснення pyproject.toml

pyproject.toml — сучасний стандарт конфігурації Python-проєкту (PEP 518 / PEP 621).

- [project] — метадані пакета: name, version, description, requires-python.
- [build-system] — інструмент збірки. Використовується setuptools>=68 із backend setuptools.build_meta.
- [tool.setuptools.packages.find] з where = ["src"] — вказує, що пакети лежать у src/, а не в корені. Це і є **src-layout**.

Завдяки цьому файлу працює команда:

bash

```
pip install -e .
```

яка встановлює пакет у режимі development («editable install»), і модуль data_processor стає доступним для імпорту звідусіль.

---

9. Вміст .gitignore

gitignore

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.egg-info/
.eggs/
build/
dist/

# Virtual environments
.venv/
venv/
env/

# IDE
.vscode/
.idea/
*.swp
```

OS
.DS_Store
Thumbs.db

Lab reports / personal
*.pdf
Файл гарантує, що в репозиторій **не потрапляють**: кеш Python, віртуальне середовище, збірки, файли IDE та OS.

---

10. Фрагмент README
markdown
Lab 02 — Banking Accounts Analysis

Лабораторна робота №2 з Python.
Варіант 4 — аналіз системи банківських рахунків.

Можливості

- list / tuple / set / dict
- Comprehensions (list / set / dict)
- collections: Counter, defaultdict, deque
- Пошук, фільтрація, групування, сортування, агрегація
- **args,* **kwargs, lambda, closures, decorators
- Benchmark: list search vs dict-index search

Запуск

```bash
python -m venv .venv
# *Windows:*
.venv\Scripts\activate
# *Linux/macOS:*
source .venv/bin/activate

pip install -e .
python -m data_processor.main
```

Без встановлення:

```powershell
# Windows PowerShell
$env:PYTHONPATH = "src"; python -m data_processor.main
```

---

11. Screenshots запуску застосунку

Нижче — опис того, що потрібно додати у звіт як скріншоти (у реальному документі тут вставляються самі зображення).

Скріншот 1. Термінал після активації venv та pip install -e .:

text

```
(.venv) PS C:\3YEAR\PYTHON\project> pip install -e .
Obtaining file:///C:/3YEAR/PYTHON/project
  Installing build dependencies ... done
  ...
Successfully installed data-processor-0.1.0
```

Скріншот 2. Запуск програми python -m data_processor.main:

text

```
============================================================
LAB 02 — Banking accounts analysis (Variant 4)
============================================================
  [repeat-decorator] banking data processing...
  [repeat-decorator] banking data processing...

============================================================
1) Built-in structures: list / tuple / set / dict
============================================================
ACCOUNT_TYPES (tuple)  : ('current', 'savings', 'deposit', 'credit')
CURRENCIES (set)       : {'EUR', 'USD', 'UAH'}
accounts (list) len    : 10
sample dict            : {'id': 1, 'client': 'Anna Koval', ...}

...

============================================================
10) Benchmark: list search vs dict search
```

```
==========================================================================
============
 records | queries |  index build |       linear |        dict
--------------------------------------------------------------------
     500 |     200 |     0.000071 |     0.003101 |     0.000008
    5000 |     200 |     0.000680 |     0.030044 |     0.000009
   20000 |     200 |     0.002601 |     0.121143 |     0.000010

[LOG ] -> demo_logged_summary
[LOG ] <- demo_logged_summary
```

---

12. Результат git status

Після першого коміту:

powershell

```
PS C:\3YEAR\PYTHON\project> git status
On branch main
nothing to commit, working tree clean
```

Після внесення змін (до коміту):

powershell

```
PS C:\3YEAR\PYTHON\project> git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
        modified:   src/data_processor/analytics.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        docs/report.md
```

Файли .venv/ та __pycache__/ у виводі **відсутні** завдяки .gitignore.

---

13. Результат git log --oneline

text

```
a17c2d8 (HEAD -> main, origin/main) Add README and .gitignore
5b9e401 Add benchmark section to analytics
c3f0a12 Implement closures and parametrized decorator
1e8b0a7 Add processors: filter, group, search
8d2c6f4 Initial commit: project skeleton and data module
```

Історія відображає логічні етапи розробки: від скелету — до повної реалізації.

---

14. Відповіді на контрольні запитання

1. Що таке list**?**
Впорядкована змінна (mutable) колекція довільної кількості елементів, доступ за індексом, часова складність доступу — O(1).
2. Чим list **відрізняється від** tuple**?**
list — змінний, tuple — незмінний. Tuple швидший і може бути ключем у dict/елементом set.
3. Чому tuple **називається immutable?**
Після створення кортежу його елементи не можна змінити, додати чи видалити — така властивість гарантує безпеку даних та хешованість.
4. Для чого використовується set**?**
Для зберігання унікальних елементів і швидких операцій над множинами (об'єднання, перетин, різниця, симетрична різниця).
5. Яка властивість елементів set **є важливою?**
Hashability — елементи повинні бути незмінними (хешованими).
6. Що таке dict**?**
Впорядкований набір пар «ключ → значення» з доступом за ключем за середній час O(1).
7. Що таке key-value structure?
Структура даних, де кожному ключу відповідає значення (наприклад, dict).
8. Чому пошук key у dict **зазвичай швидкий?**
Тому що dict реалізовано через хеш-таблицю: ключ хешується → обчислюється бакет → доступ за O(1) у середньому.
9. Що таке hashability?
Властивість об'єкта мати незмінний хеш протягом життя — необхідна умова для використання як ключа dict або елемента set.
10. Чому list **не може бути key у** dict**?**
Тому що list — змінний (mutable) і не має стабільного хешу.
11. Для чого використовується Counter**?**
Для підрахунку частоти елементів у колекції.
12. Для чого використовується defaultdict**?**
Автоматично створює значення за замовчуванням для нового ключа — зручно для групування.
13. Для чого використовується deque**?**
Двостороння черга з O(1) додаванням/видаленням на обох кінцях.
14. Що таке list comprehension?
Компактний синтаксис створення списку: [x for x in iterable if cond].
15. Що таке set comprehension?
Аналогічно, але результат — set: {x for x in iterable}.
16. Що таке dict comprehension?
Створення словника: {k: v for k, v in items}.

17. Що таке generator expression?
Лінивий ітератор у дужках: (x for x in iterable) — не створює проміжного списку.
18. Що означає O(1)**?**
Константний час — не залежить від розміру даних.
19. Що означає O(n)**?**
Лінійний час — пропорційний кількості елементів.
20. Що означає O(n log n)**?**
Час типовий для ефективних алгоритмів сортування.
21. Що означає $O(n^2)$**?**
Квадратичний час — характерний для подвійних циклів по всіх елементах.
22. Яка приблизна складність пошуку у list**?**
O(n) — лінійний пошук.
23. Яка середня складність пошуку у set**?**
O(1) у середньому.
24. Яка середня складність пошуку key у dict**?**
O(1) у середньому.
25. Чому правильна структура даних може прискорити програму?
Бо операції в ній мають меншу асимптотичну складність (наприклад, пошук у dict замість list: O(1) проти O(n)).
26. Що таке функціональна декомпозиція?
Розбиття програми на невеликі функції, кожна з яких виконує одну чітко визначену задачу.
27. Які переваги малих functions?
Читабельність, тестованість, повторне використання, легший рефакторинг.
28. Що таке positional argument?
Аргумент, що передається за позицією: f(1, 2).
29. Що таке keyword argument?
Аргумент, що передається за іменем: f(a=1, b=2).
30. Для чого використовується *args**?**
Приймає довільну кількість позиційних аргументів у вигляді tuple.
31. Для чого використовується **kwargs**?**
Приймає довільну кількість іменованих аргументів у вигляді dict.
32. Що таке lambda **function?**
Коротка анонімна функція, що складається з одного виразу.
33. Де доцільно використовувати lambda**?**
Як короткі «key»-функції в sorted, max, min, filter, map.
34. Чому складну business logic не варто записувати в lambda**?**
Через погану читабельність, неможливість багаторядкових виразів та ускладнене тестування.
35. Що таке closure?
Функція, що зберігає доступ до змінних зовнішньої функції навіть після її завершення.

36. Які variables може пам'ятати closure?
Будь-які локальні змінні зовнішньої функції, що використовуються у внутрішній.
37. Що таке decorator?
Функція вищого порядку, що приймає функцію та повертає нову функцію, розширюючи її поведінку без зміни коду.
38. Для чого застосовуються decorators?
Логування, вимірювання часу, кешування, авторизація, валідація — наскрізна функціональність.
39. Для чого використовується functools.wraps**?**
Зберігає метадані оригінальної функції (__name__, __doc__) у обгортці.
40. Чому потрібно оцінювати складність алгоритму?
Щоб передбачати поведінку програми при зростанні даних та обирати ефективні рішення.

---

15. Аналіз складності (≥ 3 операції)

Операція	Структура	Складність
Пошук рахунку за account_number у списку	list	**O(n)**
Пошук за account_number у dict-індексі	dict	**O(1)** середня
Побудова dict-індексу	dict	**O(n)**
Перевірка currency in CURRENCIES	set	**O(1)** середня
Сортування за balance	sorted	**O(n log n)**
Сума балансів через генератор	—	**O(n)** без проміжного списку

Результати бенчмарку

records	queries	index build	linear search	dict search
500	200	0.000071 s	0.003101 s	0.000008 s
5 000	200	0.000680 s	0.030044 s	0.000009 s
20 000	200	0.002601 s	0.121143 s	0.000010 s

Висновок з бенчмарку: час лінійного пошуку зростає пропорційно n (підтверджує O(n)), тоді як час пошуку через dict-індекс залишається практично сталим (підтверджує O(1) у середньому). Це емпірично підтверджує теоретичні оцінки.

16. Висновок

У ході лабораторної роботи №2 було створено модуль обробки структурованих даних для системи банківських рахунків (Варіант 4). Реалізовано всі вимоги завдання:

- **Базові структури:** list, tuple, set, dict — кожна використана за призначенням: list[dict] для основного датасету, tuple для переліку типів рахунків, set для валют, dict для індексів.
- **Модуль** collections**:** використано Counter для підрахунку, defaultdict для групування, deque для історії операцій.
- **Comprehensions:** list, set та dict comprehension, а також генераторні вирази для агрегації без проміжних списків.
- **Функціональна декомпозиція:** код розбито на модулі data, processors, analytics, decorators, main — кожен з чіткою відповідальністю.
- **Функції вищого порядку:** filter_items, sort_items, pipeline.
- *args **і** **kwargs**:** реалізовано у calculate_average_values і create_record.
- lambda**:** використано у сортуваннях, пошуку максимуму та pipeline.
- **Closure:** create_min_balance_filter, create_balance_range_filter.
- **Decorator:** log_call, measure_time, параметризований repeat(count).
- **Оцінювання складності:** проаналізовано 6 операцій; емпірично підтверджено різницю між O(n) і O(1) у бенчмарку.

Практична цінність: проєкт демонструє перехід від простого використання колекцій Python до усвідомленого вибору структур даних і побудови модульної логіки обробки. Модуль можна використати як компонент наскрізного навчального проєкту.

Навички, отримані під час роботи:

- проєктування Python-пакета у src-layout;
- використання pyproject.toml та pip install -e .;
- написання ідіоматичного Python з type hints;
- робота з Git (init, add, commit, push, .gitignore);
- оцінювання алгоритмічної складності та профілювання.